Практична робота №3.1. Дослідження систем контролю версій програмного проєкту
(Робота з GIT (контроль версій))

Завдання для практичного виконання

1. Ознайомитися з ГІТ - http://githowto.com/. Необхідно навчитися додавати файли в репозиторій, комітити, пушити, пулити, вернути коміт, вернути запушений коміт, створити гілку, закомітити гілку, запушити гілку.
Про Git українською https://git-scm.com/book/uk/v2
GIT Quiz- практикум по использованию git, для начинающих

2. Покласти свій сайт під GIT.
Необхідно щоб Ваш сайт знаходився під GITом (https://github.com/).
Зареєструйтеся в системі, створіть власний проект, завантажте до нього раніше створені файли.
Включитися до колективного проекту. Освоїти команди по роботі з репозитарієм.

3. Осягнути суть GIT та зрозуміти навіщо він потрібен.
Початок роботи з GIT.

4. Осягнути, що таке GIT-flow. Навчитись використовувати.
4. Виконані Вами дії з елементами теоретичних відомостей оформіть як звіт та захистіть у викладача.
Теорія
Що таке репозиторій коду і для чого він потрібен?
Уявіть, що ви щойно закінчили написання своє суперської бібліотеки, яка вміє робити чергову круту штуку і тепер бажаєте закинути це все на продакшин сервер, щоб кінцеві користувачі могли нею користуватись.
Що ви для цього робили б, якщо не було репозиторію коду?
Заархівували якимось чином увесь ваш код (файлики та папки), закинули на віддалений продакшин сервер (через ssh або ftp), розпакували і поклали у потрібне місце, щоб новий код вступив у дію.
Тепер уявіть, що випадково ви поредагували код на продакшині, щось поламали і тепер треба це пофіксити.
Що ви для цього робили б без репозиторію коду? Ще раз розпакували попередній архів і надписали поламані файлі файлами з архіву.
Ще уявіть, що ви зробили зміни до вашого коду, але потім передумали і захотіли все повернути. Без репозиторія коду, у кращому випадку, вам прийшлось би використовувати функцію вашого редактора (якщо зміни не розтягнуті у кілька днів чи тижнів) Відкотити Зміни. У гіршому – ще раз переписувати код.
А тепер уявіть ще один сценарій. Над вашою бібліотекою окрім вас працює ще 10 ваших друзів. Одночасно. Кожен працює над власним модулем (файликом) чи пакетом (цілою папочкою коду), а дехто працює над різними частинами коду в межах одного файлика. Що робити, коли усі закінчили власну частину коду?
Хтось один сідає, збирає архів бібліотеки від 10-ти розробників та об'єднує усі архіви в один. При цьому намагаючись порівняти чи часом не були змінені однакові рядочки коду різними розробниками.
І на засипку. Цей нещасний, хто збирав усі архіви докупи, побачив, що бібліотека чомусь не працює, навіть не запускається. Розпитує кожного із розробників про потенційну причину. А кожен відповідає, що у нього працювало і що це мабуть проблема у неправильному об'єднанні усіх архівів коду.

Щоб уникнути подібних сценаріїв придумали репозиторії коду. **Репозиторій коду – це програмний інструмент, основне завдання якого – зберігання коду та історії змін до даного коду.** Також його ще називають **Системою Контролю Версій** (**VCS**). Маються на увазі версії коду.

Що репозиторій нам дає? З вище наведених жахливих сценаріїв можемо здогадатися, що репозиторій полегшує життя розробника у кожному із них. А саме, репозиторій коду дозволяє:

- зберігати ваш код;
- запам'ятовувати історію змін до вашого коду (та дозволяти у будь-який момент побачити хто саме зробив зміни, коли зробив зміни);
- та відкотити до будь-якої версії коду у будь-який момент;
- з легкістю об'єднувати зміни різних версій, станів та розробників;
- що приводить нас до наступного пункту: розробляти проект командою розробників, одночасно працюючи над одними і тими ж модулями і навіть стрічками коду;
- готувати ваш код до релізів на продакшин системи та публікувати ваш код для зовншінього світу;
- вести статистику змін до коду;
- і ще багато інших класних штук!

Одним словом, в сучасному IT світі без репозиторія нікуди. В більшості випадків хоча б одним із рипозиторіїв повинні володіти не лише програмісти, але й і верстальщики, QA-ї і інші спеціалісти, які працюють у вашій команді та мають доступ до коду, тестів та документації проекту.

Тому, тепер давайте до діла. Які бувають типи репозиторіїв:

Види Репозиторіїв Коду

Є два типи репозиторіїв.

1. Централізовані Репозиторії Коду (Version Control System, VCS)

Старіші та простіші, які зараз практично не використовуються у нових проектах. Приклад: SVN, CSV.

Основна їхня ознака – може бути лише один центральний репозиторій коду для проекту, куди усі розробники комітять свій код. Якщо репозиторій на зовнішньому сервері і у вас немає інтернету – ви працювати у гнучкому режимі (робити часті записи у репозиторій) не зможете.

Я коли починав кодувати користувався репозиторієм SVN. Простий та зрозумілий у користуванні, але важка робота з гілками + як я вже згадав, усі репозиторії даного типу не підтримують більше, ніж один центральний репозиторій.

2. Розподілені Репозиторії Коду (Distributed Version Control System, DVCS)

На щастя з часом придумали так звані Розподілені (англ. Distributed) репозиторії коду, які дозволяють нам, окрім центральльного репозиторія, також мати кожному розробнику власний локальний репозиторій, і взагалі скільки завгодно копій/клонів репозиторію і працювати з кожним із них автономно. Деколи навіть важко розібратися, а який з них центральний 😉" class="wp-smiley">

Таким чином для регулярних комітів вам не потрібен постійний доступ до інтернету. Ви можете робити записи у локальний репозиторій, а вже коли з'явиться інтернет – закинути усі зміни на віддалені необхідні репозиторії.

Найбільш поширеними розподіленими репозиторіями коду вважаються Git та Mercurial. Якщо використовуєте Git, тоді також можете користуватися онлайн сервісом для своїх публічних чи приватних репозиторіїв github.com. Якщо ж Mercurial, тоді його підтримку знайдете у сервісі bitbucket.com.

На мою думку github.com на порядок популярніший. На даний час у своїй роботі використовую лише Git репозиторій (приватні сервери для клієнтських проектів) та github.com для OpenSource проектів.

Концепція та Термінологія Git

Якщо зовсім коротко, то:

Ви працюєте у своїй Робочій Папці (workspace), потім закидаєте ваші зміни у так званий Перевалочний Пункт (index або staging), звідки кладете код у ваш Локальний Репозиторій (local repository), і вже при потребі, у Віддалені Репозиторії (remote repository), щоб інші також мали доступ до ваших змін.

Тобто локальний репозиторій складаєтьс із трьох так званих дерев (tree):

1. робоча папочка (working space) – там ви працюєте,
2. індекс (staging, index) – перевалочний пункт між робочою папкою та локальним репозиторієм,
3. head – вказівник на ваш останній коміт в локальному репозиторії.

Тепер давайте розберемося детальніше в термінах:

Робоча Папка (Working Directory) – це ваша локальна папка у файловій системі, яка відображає певний стан коду в репозиторії + ваші локальні зміни (якщо такі є). Ви весь час працюєте над кодом будучи всередині вашої робочої папки.

Коміт (commit) – запис змін коду у репозиторій.

Мердж (merge) – об'єднання змін з однієї гілки коду в іншу.

Гілка (branch) – версія коду, яку створють всередині того ж репозиторію, і ведуть розробку зазвичай до моменту поки не зроблять мердж (об'єднання) змін даної версії коду із основною гілкою (зазвичай master). Гілки використовують для роботи над окремими завданнями одночасно, не заважаючи один одному. Дефолтна гілка у будь-якому git репозиторії – це *master*.

Пул (pull) – отримання останніх змін з віддаленого репозиторію у ваш власний локальний репозиторій.

Push (push) – закинути зміни локального репозиторію у віддалений репозиторій.

Ремоут (remote) – один із віддалених репозиторіїв. Один код може мати багато клонів та копій на вашому та зовнішніх серверах. Кожна така копія, окрім вашої локальної, називається ремоутом (віддаленим репозиторієм).

Індекс (Stage або Index) – підготовче місце ваших файлів з робочої папки перед тим як закинути зміни робочої папки у локальний репозиторій. Зазвичай коміт включає у себе лише ті зміни, що є в індексі. Давайте для простоти називати дане місце Перевалочним Пунктом, кращого терміну поки не придумав.

Локальний репозиторій – ваша власна копія репозиторію, у яку безпосередньо йдуть коміти з вашої робочої папочки (насправді все йде через Перевалочний Пункт – Stage (Index)). Для роботи з даним репозиторієм вам не потрібно інтернету.

Git HEAD – вказівник на коміт в репозиторії, на якому зараз перебуває ваш локальний репозиторій і гілка у цьому репозиторії. Вказівник по дефолтну вказує на останній коміт, але його можна вручну "переміщати" між комітами, гілками і навіть тегами (у цьому випадку його називають Detached Head – від'єднаний вказівник).

Тег (tag) – фіксація стану коду для подальшого використання його при релізах. Фіксуємо зазвичай код під тегом з певною назвою, як от:*1.0*, *2.0.1*, *3.0alpha*. Нові зміни у теги не записуються! Тому вони ще носять назву *Фіксовані Гілки*.

Встановлюємо Git на свій комп'ютер

Взалежності від того, яку операційну систему ви використовуєте обираєте один із наступних варіантів завантаження Git системи:

- Інтсталюємо Git на Макінтоші
- Інсталюємо Git на Віндовсі
- Інсталюємо Git на Лінуксі

Кожна із вище наведених інсталяцій включає у себе графічний інтерфейс. Тому, якщо ви більше любите користуватись мишкою, ніж “колупатись” в командній стрічці, можете попрактикуватись з графічними клієнтами.

Конфігуруємо Git

Мінімальна конфігурація Git репозиторію включає налаштування власного імені (воно згадуватиметься в кожному вашому коміті, так само як і ваш емейл):

$ git config user.name "Vitaliy Podoba"

та власної емейл адреси:

$ git config --global user.email "example@gmail.com"

Зауважте використання опції “-global”. Вона встановлює ваші дані глобально для усіх репозиторіїв на вашому комп’ютері. Якщо для певного репозиторію вам потрібно інші ваші дані встановити, тоді заходите у папочку репозиторію, забираєте опцію global і ваші дані будуть встановлені локально лише для поточного репозиторію.

Також варто одразу встановити список файлів, які git повинен ігнорувати. Зазвичай усі генеровані файли не повинні попадати в репозиторій. Для цього створюю файл .gitignore_global будь-де у себе на комп’ютері. Ось приклад:

```
*.pyc
*~
*.egg-info
*.mo
*.egg
```

Даний файл заставить git ігнорувати файлики, що закінчуються на .pyc, ~, .egg-info і т.д. Більшість має безпосереднє відношення до розробки на мові Python. Під ваше конкретне розробницьке середовище вам може знадобитись кілька додаткових типів файлів включити у даний файл.

І даємо вказівку git-у записати даний список шаблонів у ігнор:

$ git config --global core.excludesfile /path/to/.gitignore_global

Опцій для конфігурації репозиторію багато, але необхідний мінімум ми оглянули.

Клонуємо існуючий репозиторій

Найпростіший випадок. Десь уже існує репозиторій з кодом, і вам треба його собі скопіювати для подальшої роботи:

$ git clone https://github.com/vipod/wxpython_calculator.git

Знаючи посилання на репозиторій коду можемо скористуватися командою clone, щоб отримати локальний репозиторій (клон) і робочу папочку + перевалочний пункт. Все необхідне для початку роботи з кодом.

В результаті вище описаної команди у вас локально з’явиться папка wxpython_calculator з кодом всередині. А також службова папка під назвою ‘.git’. Не рекомендую туди заходити і щось змінювати без додаткових знань, які в даній статті не описані.

Створюємо новий репозиторій

Створюємо локально папочку, заходимо всередину та ініціалізуємо її як git репозиторій:

створюю папку

$ mkdir myrepo

заходжу в папку

$ cd myrepo

ініціюю її як git репозиторій

$ git init

Закидуємо локальні зміни у Локальний Репозиторій

Продовжуємо роботу з нашою щойно створеною та ініціалізованою папочкою.

Створюємо всередині необхідні файли (модулі) з кодом, підпапки при необхідності і додаємо їх усіх у Індекс – перевалочний пункт перед комітом (записом у локальний репозиторій):

$ git add *

Команда git add додасть рекурсивно усі файли та підпапки даної папки.

Зауважте, у лінуксових командних стрічках команда git add може поскаржитись, що деякі файлики містять не лінуксовий, а DOS закінчення файлів. У такому випадку скористайтесь утилітами як от dos2unix для переформатування проблемних файлів, і повторно спробуйте команду git add аж допоки усі необхідні файли не опиняться у нашому Перевалочному Пункті.

Також рекомендую завжди перед тим, як закидувати файли в Перевалочний Пункт перевіряти, що саме ви закидуєте. Команда status:

```
$ git status
# On branch master
# Changed but not updated:
#   (use "git add <file>..." to update what will be committed)
#   (use "git checkout -- <file>..." to discard changes in working directory)
#
#    modified:   calculator.py
#
# Untracked files:
#   (use "git add <file>..." to include in what will be committed)
#
#    test.txt
no changes added to commit (use "git add" and/or "git commit -a")
```

Ця команда показує на якій гілці коду ви знаходитесь. У даному випадку це дефолтна гілка – master. А також список змінених, нових та видалених файлів. Також дана команда дає підказки, що і з кожним із даних файлів можна робити далі.

У даному випадку команда показує, що новий файл з’явився test.txt, а також був змінений calculator.py.

Закидуємо зміни з Індекса (staging) у Локальний Репозиторій. Це якраз і називається Зробити Коміт:

$ git add calculator.py test.txt
$ git commit -m 'initial code version'
У вище наведеному прикладі ми додали усі змінені та нові файли до індексу. А потім закомітили стан індексу в локальний репозиторій командою commit.

Щоб зменшити кількість команд для коміту і пропустити Перевалочний Пункт та одразу закинути ваші локальні зміни у Локальний репозиторій, можна скористатись опцією '-a' команди commit, яка позбирає усі змінені файли та включить їх у коміт:

$ git commit -a -m 'fix issue #132'

Зауважте, що нові файли чи видалені не будуть враховуватись опцією '-a'. Для нових файлів потрібно їх додавати окремою командою:

$ git add test.txt

І ще, якщо був видалений файл, то ці зміни також треба занести в Індекс окремою командою, інакше Локальний Репозиторій проігнорує видалення вашого файлу у вашій робочій папці:

$ git rm removed_file.txt

І на закінчення даної секції. Якщо ви в робочій папочці зробили випадкові зміни у файл, або просто хочете відкотити локальні зміни до стану такого як в репозиторії. Або якщо потрібно відновити випадково видалений файл у робочій папці, тоді користуйтесь наступною командою:

$ git checkout -- removed_file.txt

Ця команда відновить стан даного файлу у такий самий, яким він є у Локальному Репозиторії. Її також можна застосовувати до цілої папки одразу.

Закидуємо зміни Локального Репозиторію у Віддалені

Зміни наші пішли щойно у Локальний репозиторій. Але тепер, щоб інші могли користуватись нашими оновленнями, нам треба їх закинути у один із існуючих зовнішніх репозиторіїв (ремоутів).

Дефолтний віддалений репозиторій називається origin. Якщо ви початково клонували існуючий репозиторій, тоді origin у вас уже налаштований разом з гілкою коду, звідки ви клонували. У такому випадку наступної команди буде достатньо, щоб синхронізувати ваш Локальний Репозиторій з Віддаленим:

$ git push origin master

Ця команда закидає усі коміти Локального Репозорію поточної гілки у Віддалений репозиторій під назвою origin в гілку master (дефолтна гілка).

Зазвичай при клонуванні репозиторію ремоут + гілка уже запам'ятовуються для вас, тому достатньо наступної команди:

$ git push

Якщо ж ми хочемо закинути на зовні наш новостворений Локальний репозиторій, тоді треба спочатку додати дефолтний ремоунт і вказати його адресу:

$ git remote add origin http://your-git-server.com/some_repository_name

Ми сказали нашому репозиторію локальному, що під адресою http://your-git-server.com/some_repository_name знаходиться наш віддалений репозиторій під назвою origin.

Тепер можемо тією ж командою закинути зміни у даний репозиторій у дефолтну гілку master:

$ git push origin master

Ок. Маємо 2 репозиторії у синхронізованому стані: локальний та origin.

Оновлюємо локальний репозиторій та робочу папку

Уявимо, що наш товариш зробив зміни у код та закинув із власного локального репозиторію у віддалений. І тепер ми хочемо отримати його зміни у свій локальний репозиторій, ну і потім відповідно у робочу папочку, щоб працювати над актуальною версією коду.

Для цього можна скористатися однією командою:

```
$ git pull origin master
# коротший варіант, якщо git уже запам'ятав для вас ремоут та гілку
# і вони вам підходять
$ git pull
```

Ця команда складається із двох етапів:

- синхронізує віддалений репозиторій із вашим локальним ($ git fetch)
- мерджить (об’єднує) нові зміни, що прийшли щойно в локальний репозиторій, із вашою робочою папочкою ($ git merge)

Ці два етапи можна виконувати самому, окремо у вигляді вище наведених в дужках командах.

Робота з Тегами

Кожного разу, коли потрібно робити новий реліз коду, вартує користуватися Тегами. Тег – це зафіксована незмінювана версія коду позначена певним іменем версії (напр. 1.0.2), що використовується для релізів та продакшин деплойментів.

Щоб переглянути список існуючих тегів:

$ git tag -l

Щоб створити новий тег (анотований, є й інші види, але їх тут ми не розглядаємо):

$ git tag -a 1.0.1 -m 'release 1.0.1'

Даною командою створюємо новий тег, називаючи його 1.0.0 та додаючи супроводжуючий коментар ‘release 1.0.1′ – коментар також обов’язковий аргумент.

Щоб видалити існуючий тег з Локального репозиторію:

$ git tag -d 1.0.1

Щоб закинути усі теги і синхронізувати їз з Локального на Віддалений репозиторій:

$ git push --tags

Тобто команда git tag створює тег лише в локальному репозиторії, а вже push знову нам допомагає, щоб закинути теги у необхідний список зовнішніх репозиторіїв.

Додаткові Матеріали

В даній статті наведені далеко не всі можливості, а лише необхідний мінімум для вашої щоденної роботи з репозиторієм. Для подальшої інформації наводжу кілька лінків на корисні Git матеріали:

- шпаргалка по Git;
- Pro Git книга на російській, безкоштовна;
- хакерський ввід у Git англійською: http://wildlyinaccurate.com/a-hackers-guide-to-git ;
- ази Git англійською, коротко і ясно: https://rogerdudler.github.io/git-guide/ .

Питання, які необхідно опрацювати студентові самостійно:

- Що таке GIT

- Можливості GIT